

Supplementary Material

for the paper

“Counting HyperGraphlets via Color Coding: a Quadratic Barrier and How to Break It”

Marco Bressan
University of Milan
Milan, Italy
marco.bressan@unimi.it

Stefano Clemente
University of Milan
Milan, Italy
stefano.clemente@unimi.it

Giacomo Fumagalli
University of Milan
Milan, Italy
giacomo.fumagalli@unimi.it

Contents

1	Proofs	2
1.1	Lemma 5	2
1.2	Lemma 6	2
1.3	Lemma 7	2
1.4	Lemma 8	3
1.5	Lemma 9	4
1.6	Lemma 10	4
1.7	Corollary 1	5
1.8	Theorem 1	5
2	Additional Experiments	10
2.1	Speedup and absolute build-up timings	11
2.2	Hypergraphlet Frequencies	13
2.3	Parallel scalability	15
2.4	Accuracy	18

1 Proofs

This section contains the proofs of results stated in the main paper that were omitted due to space constraints.

1.1 Lemma 5

Lemma 5. *NW-IE solves NW in time $O(2^{\Delta(H)} \cdot |V|)$.*

Proof. Fix $v \in V$ and $X \subseteq E(v)$. Note that line 4 increments $t[X]$ by $w(v)$ if and only if $v \in \cap_{e \in C} e$. Therefore, at line 5 we have:

$$t[X] = \sum_{v \in \cap X} w(v) = w\left(\bigcap X\right). \quad (1)$$

By construction, then, the second loop ensures that for any $v \in V$:

$$\eta(v) = \sum_{\emptyset \neq X \subseteq E(v)} (-1)^{|X|+1} t[X] - w(v) \quad (2)$$

$$= \sum_{\emptyset \neq X \subseteq E(v)} (-1)^{|X|+1} w\left(\bigcap X\right) - w(v) \quad \text{by Eq. (1)} \quad (3)$$

$$= w\left(\bigcup E(v)\right) - w(v) \quad \text{by I.-E. (Eq. (14))} \quad (4)$$

For the time complexity, for every $v \in V$ the algorithm performs at most $2^{|E(v)|} \leq 2^{\Delta(H)}$ operations, each of which takes time $O(1)$. $\square$

1.2 Lemma 6

Lemma 6. *If H is (α, β) -nice, Algorithm 3 computes the counters of Equation (4) in time $2^{O(k)} \cdot (2^\beta |V| + \alpha^2 \beta \|H\|)$.*

Proof. The correctness follows from the discussion above, which shows that at line 19 we have $\eta(v) = \mathcal{C}(T_2, S_2, v)$, from $\mathcal{C}(T, S, v) = \frac{1}{d} \sum_{\substack{S_1, S_2 \subseteq S \\ S_1 \cap S_2 = \emptyset}} \mathcal{C}(T_1, S_1, v) \sum_{u \sim v} \mathcal{C}(T_2, S_2, u)$, and by construction of the algorithm. Let us bound the running time.

By Lemma 4, line 5 takes time $O(\|H\|)$. By the discussion above, line 6 and line 7 take time $O(\alpha^2 \beta \|H\|)$. The same discussion showed that each execution of the loop of line 14 takes time $O(\alpha^2 \beta \|H\|)$ as well, while line 11, line 12 and line 13 take time respectively $O(|V|)$, $O(\alpha^2 |E|)$, and $O(2^\beta |V|)$. Thus, every iteration of the loop of line 10 takes time

$$O\left(2^\beta |V| + \alpha^2 |E| + \alpha^2 \beta \|H\|\right) = O\left(2^\beta |V| + \alpha^2 \beta \|H\|\right). \quad (5)$$

which already dominates the running time of the rest. Noting that the said loop makes $2^{O(k)}$ iterations completes the proof. $\square$

1.3 Lemma 7

Lemma 7. *Given as input (T, S, v) , SAMPLENEIGH returns (T_2, S_2, u) such that $u \in N(v)$ with probability proportional to $C(T_2, S_2, u)$.*

Proof. Consider any iteration of the loop of line 4. Let $p(u)$ be the probability, ignoring line 12, that the iteration draws the particular vertex u . We shall prove that:

$$p(u) \propto \mathcal{C}(S_2, u) \cdot (\mathbb{1}\{u \in N_{\leq \alpha}(v)\} + \mathbb{1}\{u \in N_{> \alpha}(v)\}) \quad (6)$$

By line 12, this implies that every single iteration draws *and* accepts u with probability proportional to $\mathcal{C}(T_2, S_2, u)$, proving the claim.

Let $p_{\leq \alpha}(u)$ be the probability that an execution of line 6 draws u , and $p_{> \alpha}(u)$ the probability that an execution of the block of line 7 draws u . By the law of total probability:

$$p(u) = p(N_{\leq \alpha}(v)) \cdot p_{\leq \alpha}(u) + p(N_{> \alpha}(v)) \cdot p_{> \alpha}(u) \quad (7)$$

$$= \frac{W_{\leq \alpha}(v) \cdot p_{\leq \alpha}(u)}{W_{\leq \alpha}(v) + W_{> \alpha}(v)} + \frac{W_{> \alpha}(v) \cdot p_{> \alpha}(u)}{W_{\leq \alpha}(v) + W_{> \alpha}(v)} \quad (8)$$

Let us then analyse $p_{\leq \alpha}(u)$ and $p_{> \alpha}(u)$ separately.

For $p_{\leq \alpha}(u)$, line 6 implies:

$$p_{\leq \alpha}(u) = \frac{\mathcal{C}(S_2, u)}{W_{\leq \alpha}(v)} \cdot \mathbb{1}\{u \in N_{\leq \alpha}(v)\} \quad (9)$$

For $p_{> \alpha}(u)$, let $p_{inn}(u)$ be the probability that, ignoring line 11, one iteration of line 8 draws a particular vertex u ; let $p(e)$ be the probability that one execution of line 9 returns the specific hyperedge e ; and let $p(u|e)$ be the probability that, conditional on this event, one execution of line 10 returns u . By the law of total probability:

$$p_{inn}(u) = \mathbb{1}\{u \in N_{> \alpha}(v)\} \cdot \sum_{e \in E(v)} p(u | e) p(e) \quad (10)$$

$$= \mathbb{1}\{u \in N_{> \alpha}(v)\} \quad (11)$$

$$\cdot \sum_{e \in E(v)} \frac{\mathcal{C}(S_2, u)}{\mathcal{C}_2(S_2, e)} \frac{\mathcal{C}(S_2, e)}{\sum_{e' \in E(v)} \mathcal{C}(S_2, e')} \cdot \mathbb{1}\{u \in e\} \quad (12)$$

$$= \mathbb{1}\{u \in N_{> \alpha}(v)\} \cdot \sum_{e \in E(v)} \frac{\mathcal{C}(S_2, u)}{\sum_{e' \in E(v)} \mathcal{C}(S_2, e')} \cdot \mathbb{1}\{u \in e\} \quad (13)$$

$$= \mathbb{1}\{u \in N_{> \alpha}(v)\} \cdot \frac{\mathcal{C}(S_2, u) |E(v) \cap E(u)|}{\sum_{e' \in E(v)} \mathcal{C}(S_2, e')} \quad (14)$$

By line 11, this implies that every single iteration draws and accepts u with probability proportional to $\mathcal{C}(S_2, u) \cdot \mathbb{1}\{u \in N_{> \alpha}(v)\}$, that is:

$$p_{> \alpha}(u) \propto \mathcal{C}(S_2, u) \cdot \mathbb{1}\{u \in N_{> \alpha}(v)\} \quad (15)$$

Since $\sum_{u \in N(v)} \mathcal{C}(S_2, u) \cdot \mathbb{1}\{u \in N_{> \alpha}(v)\} = W_{> \alpha}(v)$, we deduce that:

$$p_{> \alpha}(u) = \frac{\mathcal{C}(S_2, u)}{W_{> \alpha}(v)} \cdot \mathbb{1}\{u \in N_{> \alpha}(v)\} \quad (16)$$

Plugging $p_{\leq \alpha}(u)$ and $p_{> \alpha}(u)$ in Equation (8) proves the claim. $\square$

1.4 Lemma 8

Lemma 8. *With the random number generators precomputed as above, each invocation of `SAMPLENEIGH` takes expected time $O(\beta^2)$.*

Proof. Let I be the random variable that counts the total number of iterations executed by the while loop at line 8. It holds $I \sim \text{Geom}(|E(v) \cap E(u)|)$. Let T be the random variable that counts the total number of iterations within the body of a single execution of the while loop at line 4. Then:

$$T = \begin{cases} 1 & \text{with probability } p(N_{\leq \alpha}) \\ I & \text{with probability } 1 - p(N_{\leq \alpha}) \end{cases}$$

It follows that

$$\mathbb{E}[T] = p(N_{\leq \alpha}) + (1 - p(N_{\leq \alpha}))\mathbb{E}[I] \leq 1 + \beta \quad (17)$$

Thus, it holds $\mathbb{E}[T] = O(\beta)$. Let K be the random variable that counts total number of times the loop at line 4 is executed, and N the random variable that counts the total number of executions (including all the loops). Consider now another version of Algorithm 4 where line 12 accepts with fixed probability $\frac{1}{2}$, and let K', N' be the equivalent of K, N for this version. By a simple coupling argument, $K \leq K'$ and $N \leq N'$, and clearly K' is independent of the variables T_i . By Wald's equality, then,

$$N \leq N' = \sum_{i=1}^{K'} T_i = \mathbb{E}[K']\mathbb{E}[T] \leq 2\mathbb{E}[T] = O(\beta). \quad (18)$$

Now let us bound the time taken by the execution of every line. Outside the loop at line 4, the execution of each line takes at most $O(1)$. Drawing u at line 6 takes $O(1)$. The same holds for line 10. Sampling a hyperedge at line 9 takes time $O(1)$, whereas computing $|E(v) \cap E(u)|$ at line 11 takes time at most $O(\beta)$. Thus, if the expected number of iterations is $O(\beta)$, it follows that each invocation of `SAMPLENEIGH` takes expected time $O(\beta^2)$. $\square$

1.5 Lemma 9

Lemma 9. *By spending an additional $O(\alpha^k \cdot |E_{\leq \alpha}|)$ preprocessing time, one can compute the k -hypergraphlet $H[U]$ induced by any given k -vertex $U \subseteq V$ in time $2^{O(k)} \cdot \beta$.*

Proof. Given U , we check for each $X \subseteq U$ with $|X| \geq 2$ whether there exists $e \in E$ such that $e \cap U = X$, in which case we add X to the hyperedges of $H[U]$. We now discuss how e can be found in a generic set of edges E ; we then show how to apply the idea to $E_{\leq \alpha}$ and $E_{> \alpha}$. Suppose we have precomputed, for every $X \subseteq U$, the number $N[X] = |\{e \in E : X \subseteq e\}|$. Moreover let $N^*(X, U) = |\{e \in E : e \cap U = X\}|$; note that $N^*(X, U) > 0$ means $e \cap U = X$ for some $e \in E$. Now, by inclusion-exclusion,

$$N^*(X, U) = \sum_{Y \subseteq U \setminus X} (-1)^{|Y|} N[X \cup Y]. \quad (19)$$

Therefore, given the counters $N[X]$, we can compute $N^*(X, U)$ in time $2^{O(|U|)} \leq 2^k$.

Let us now discuss how to implement this for $E_{\leq \alpha}$ and $E_{> \alpha}$. For $E_{\leq \alpha}$, in the build-up phase for every $e \in E_{\leq \alpha}$ we enumerate all subsets $X \subseteq e$ of at most k vertices, increasing $N[X]$ accordingly. This computes the counters $N[X]$ explicitly in time $O(\alpha^k \cdot |E_{\leq \alpha}|)$. For $E_{> \alpha}$, observe that $N[X] = |\bigcup_{v \in X} E_{> \alpha}(v)|$, and the right-hand side can be computed in time $O(k\beta)$, yielding a total time of $O(2^k \cdot k\beta)$ for computing all counters once given U . By a final bound, given U , checking whether e exists in $E = E_{\leq \alpha} \cup E_{> \alpha}$ takes a total time of $2^{O(k)} \cdot \beta$, as claimed. $\square$

1.6 Lemma 10

Lemma 10. *Given a subset $U \subseteq V$ of k vertices, one can compute the adjacency matrix A_U of $\text{Gaif}(H[U])$ in time $O(k^2(\beta + \ln |V|))$.*

Proof. Clearly, $A_U[u][v] = 1$ if and only if $u \in N_{\leq \alpha}(v)$ or $E_{> \alpha}(u) \cap E_{> \alpha}(v) \neq \emptyset$. The condition $u \in N_{\leq \alpha}(v)$ can be verified in time $O(\log |V|)$ via binary search as $N_{\leq \alpha}(v)$ is sorted and has size at most $|V|$. The condition $E_{> \alpha}(u) \cap E_{> \alpha}(v) \neq \emptyset$ can be verified in time $O(\beta)$ since both sets are sorted and have size at most β . A union bound over all k^2 pairs yields the claim. $\square$

1.7 Corollary 1

Corollary 1. *Algorithm 5 takes, on each invocation, expected time $O(k\beta^2)$. Moreover, the vertex set of the output treelet is $U \in \mathcal{V}_k(H)$ with probability proportional to the number of spanning trees $\sigma(H[U])$ of $\text{Gaif}(H[U])$.*

Proof. The running time bound follows from Lemma 8 and the fact that SAMPLE makes $k - 1$ calls to SAMPLENEIGH, together with the bounds on the time for drawing (T, v) . The claim on the distribution follows from Lemma 7 and the same analysis MOTIVO¹ $\square$

1.8 Theorem 1

This section of the appendix is dedicated to the proof of Theorem 1. In particular, we prove the following equivalent statement.

Theorem 1. *There exists an FPT-reduction from $\kappa\text{-CLIQUE}$ to $\kappa\text{-SH}$ that, for every instance (G, k) of $\kappa\text{-CLIQUE}$, yields an instance (H, k') of $\kappa\text{-SH}$ with the following properties:*

- $k' = \binom{k}{2}(k+1) \leq \frac{k^3}{2}$.
- H is $(k^2, 0)$ -nice.

Moreover, the reduction runs in time $O(k^2 \cdot (|E(G)| + |V(G)|))$.

This clearly implies that $\kappa\text{-SH}$ is $W[1]$ -hard. The original claim follows from the well-known lower bound $n^{\Omega(k)}$ of $\kappa\text{-CLIQUE}$ under ETH². Note that, by the second item, $\kappa\text{-SH}$ remains $W[1]$ -hard and is subject to a $n^{\Omega(\sqrt[3]{\kappa})}$ ETH-conditional lower bound even if one takes into account the (α, β) -niceness of the hypergraph by adopting as a parameter $\kappa = k + \alpha + \beta$.

The rest of this section gives the proof of Theorem 1.

The reduction. Let (G, k) be an instance of $\kappa\text{-CLIQUE}$. We construct the corresponding instance (H, k') of $\kappa\text{-SH}$ as follows. For each vertex $v \in V(G)$, create a set $Q_v = \{a_v^1, \dots, a_v^{\binom{k}{2}}\}$, and for each edge $e \in E(G)$, create a singleton $R_e = \{c_e\}$. Define the hypergraph H by

$$V(H) = \bigcup_{v \in V(G)} Q_v \cup \bigcup_{e \in E(G)} R_e \quad (20)$$

and

$$E(H) = \{E_{uv} : e = \{u, v\} \in E(G), E_{uv} = Q_u \cup Q_v \cup R_e\}. \quad (21)$$

Finally, we set $k' := (k+1)\binom{k}{2}$, and we take (H, k') as the output instance of $\kappa\text{-SH}$.

See Figure 1 for a visual representation of the reduction. Intuitively, one can also think of this construction as encoding a weighted graph G' on the same vertex and edge set as G , where each vertex has weight $\binom{k}{2}$ and each edge weight 1.

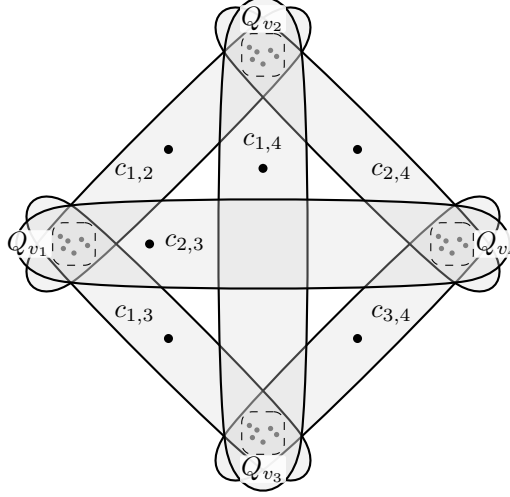


Figure 1: The hypergraph H obtained by the reduction of Theorem 1 on $G = K_4$.

We now prove that the construction above is indeed an FPT reduction that satisfies the claim of Theorem 1.

Claim 2. *The construction above runs in time $O(k^2 \cdot (|E(G)| + |V(G)|))$.*

Proof. Let (G, k) be an instance of κ -CLIQUE. For each vertex $v \in V(G)$ we create the set Q_v of size $\binom{k}{2}$, which takes a total of $O(k^2 \cdot |V(G)|)$ time. Then, for each edge $e = \{u, v\} \in E(G)$ we create the vertex c_e and the hyperedge $E_{uv} = Q_u \cup Q_v \cup \{c_e\}$, of size $2\binom{k}{2} + 1 = O(k^2)$, for a total time of $O(k^2 \cdot |E(G)|)$. Summing these two bounds gives the claimed running time $O(k^2 \cdot (|V(G)| + |E(G)|))$. $\square$

The next observation is useful to prove the correctness of the reduction. From now on, $H[U]$ refers to the section hypergraph induced by $U \subseteq V(H)$, as formalized in Definition 2.3.

Observation 1. *Let (G, k) be an instance of κ -CLIQUE and (H, k') the instance of κ -SH obtained via the reduction of Theorem 1. Let $U \subseteq V(H)$ be such that $H[U]$ is connected and $|U| \geq 2$. Then:*

- For every $v \in V(G)$, either $Q_v \cap U = \emptyset$ or $Q_v \subseteq U$.
- For every $e = \{u, v\} \in E(G)$, if $c_e \in U$ then $Q_u \cup Q_v \subseteq U$.

Proof. For the first item, suppose for a contradiction that there exists $v \in V(G)$ such that $Q_v \cap U \neq \emptyset$ but $Q_v \not\subseteq U$. Take any $x \in Q_v \cap U$ and any $y \in Q_v \setminus U$. Every hyperedge of H containing x is of the form $E_{vw} = Q_v \cup Q_w \cup R_e$, where $e = \{v, w\}$ for some neighbor w of v in G . However, since $y \notin U$, no such hyperedge E_{vw} is entirely contained in U , and therefore no hyperedge containing x appears in $H[U]$. Hence x is an isolated vertex in $H[U]$. As $|U| \geq 2$ and $H[U]$ is connected, this is impossible. Thus either $Q_v \subseteq U$ or $Q_v \cap U = \emptyset$.

For the second item, if $c_e \in U$ but some vertex of Q_u or Q_v is not in U , then the hyperedge $E_{u,v} = Q_u \cup Q_v \cup R_e$ is not entirely contained in U . Thus no hyperedge of $H[U]$ contains c_e , and c_e would be isolated in $H[U]$. Again this contradicts the connectivity of $H[U]$ (for $|U| \geq 2$), so we must have $Q_u \cup Q_v \subseteq U$. $\square$

¹Marco Bressan, Stefano Leucci, and Alessandro Panconesi. 2019. Motivo: Fast Motif Counting via Succinct Color Coding and Adaptive Sampling. *Proc. VLDB Endow.* 12, 11 (July 2019), 1651–1663. <https://doi.org/10.14778/3342263.3342640>

²Jianer Chen, Xiuzhen Huang, Iyad A. Kanj, and Ge Xia. 2006. Strong computational lower bounds via parameterized complexity. *J. Comput. System Sci.* 72, 8 (2006), 1346 – 1367. <https://doi.org/10.1016/j.jcss.2006.04.007>

Intuitively, the observation above states that U cannot "partially pick" a Q_v : as soon as U contains one vertex of Q_v , then it must contain the whole block. Similarly, U cannot contain a vertex c_e without also containing the whole blocks Q_u and Q_v .

Claim 3. *The construction above is an FPT reduction from κ -CLIQUE to κ -SH.*

Proof. We prove that (G, k) is a YES-instance of κ -CLIQUE if and only if (H, k') is a YES-instance of κ -SH. Recall that a YES-instance of κ -SH means that there exists $U \subseteq V(H)$ such that $|U| = k'$ and the section hypergraph $H[U]$ is connected.

($\Rightarrow$) From κ -CLIQUE to κ -SH. Assume that G contains a k -clique $K \subseteq V(G)$; define U as

$$U = \bigcup_{e=\{u,v\} \in E(G[K])} E_{uv} = \bigcup_{e=\{u,v\} \in E(G[K])} (Q_u \cup Q_v \cup R_e), \quad (22)$$

where $G[K]$ is the subgraph induced by K . By construction, each vertex $v \in K$ contributes $\binom{k}{2}$ vertices through Q_v and each edge $e = \{u, v\} \in E(G[K])$ contributes exactly one vertex through R_e . Clearly, since K has $\binom{k}{2}$ edges, $|U| = k \binom{k}{2} + \binom{k}{2} = (k+1) \binom{k}{2} = k'$.

We now show that $H[U]$ is connected. For every edge $e = \{u, v\} \in E(G[K])$ we have $E_{uv} \subseteq U$, hence E_{uv} is an edge of $H[U]$. In particular, for every pair of distinct vertices $u, v \in K$ there is a hyperedge E_{uv} contained in $H[U]$ that includes all vertices of Q_u , all vertices of Q_v , and the vertex c_e corresponding to $e = \{u, v\}$; thus, $H[U]$ is connected.

($\Leftarrow$) From κ -SH to κ -CLIQUE. Assume that (H, k') is a YES-instance of κ -SH. Thus, there exists $U \subseteq V(H)$ such that $|U| = k'$ and $H[U]$ is connected. Intuitively, in what follows we decompose U into whole vertex blocks Q_v and edge vertices c_e , and then use counting and connectivity arguments to show that U must correspond to exactly k such blocks that are pairwise adjacent in G , that is, to a k -clique.

First, define T and S as follows

$$T = \{v \in V(G) : Q_v \subseteq U\} \text{ and } S = \{e \in E(G) : c_e \in U\}. \quad (23)$$

Each vertex $v \in T$ contributes all $\binom{k}{2}$ vertices of Q_v to U , and each edge $e \in S$ contributes the vertex c_e . By construction of H we have

$$V(H) = \bigcup_{v \in V(G)} Q_v \cup \bigcup_{e \in E(G)} R_e, \quad (24)$$

so every vertex of U lies either in some Q_v or in some $R_e = \{c_e\}$. Moreover, by Observation 1, if $v \notin T$ then $Q_v \cap U = \emptyset$, and if $c_e \in U$ then $e \in S$. Hence

$$U = \bigcup_{v \in T} Q_v \cup \{c_e : e \in S\}, \quad (25)$$

and we can express the size of U as

$$|U| = |T| \binom{k}{2} + |S|. \quad (26)$$

Call $t = |T|$ and $s = |S|$. Using $|U| = k' = (k+1) \binom{k}{2}$, we obtain

$$(k+1) \binom{k}{2} = t \binom{k}{2} + s \implies s = (k+1-t) \binom{k}{2}. \quad (27)$$

Next, note that Observation 1 also implies that every edge $e = \{u, v\} \in S$ has both endpoints in T : indeed, since $c_e \in U$, the second item of Observation 1 gives $Q_u \cup Q_v \subseteq U$, so $u, v \in T$ by definition of T . Hence $S \subseteq \binom{T}{2}$. Let $G[T]$ be the graph with vertex set T and edge set S . By construction of H , the connected components of $H[U]$ correspond exactly to the connected components of $G[T]$: contracting each block Q_v to a single vertex v and deleting the vertices $\{c_e\}$ yields $G[T]$, and conversely every edge of $G[T]$ corresponds to a hyperedge E_{uv} in $H[U]$. Since $H[U]$ is connected, the graph $G[T]$ is connected. Therefore,

$$s \geq t - 1, \quad (28)$$

because any connected graph on t vertices has at least $t - 1$ edges. On the other hand, we clearly have

$$s \leq \binom{t}{2}. \quad (29)$$

We now derive constraints on t using Eqs. (27) to (29). In particular, we show that $t = k$, meaning that $s = \binom{k}{2}$ and thus G contains a k clique. We may assume $k \geq 3$, since for $k \leq 2$ both κ -CLIQUE and κ -SH are trivial.

Case 1: $t \geq k + 1$. From Eq. (27) we obtain $s = (k + 1 - t) \binom{k}{2} \leq 0$. However, $H[U]$ is connected and $|U| = k' > 1$, so $H[U]$ must contain at least one hyperedge, which implies $s \geq 1$. This is a contradiction. Thus $t \leq k$.

Case 2: $t \leq k - 1$. From Eq. (27) we have $k + 1 - t \geq 2$, and hence

$$s = (k + 1 - t) \binom{k}{2} \geq 2 \binom{k}{2} = k(k - 1). \quad (30)$$

On the other hand, since $s \leq \binom{t}{2}$ and $t \leq k - 1$, we have

$$k(k - 1) \leq s \leq \binom{k - 1}{2} = \frac{(k - 1)(k - 2)}{2}. \quad (31)$$

But clearly, for $k \geq 3$ we obtain

$$k(k - 1) > \frac{(k - 1)(k - 2)}{2}, \quad (32)$$

which leads to a contradiction. Therefore $t \not\leq k - 1$. The only remaining possibility is $t = k$. Finally, substituting into (27) we get

$$s = (k + 1 - k) \binom{k}{2} = \binom{k}{2}. \quad (33)$$

Since $s = |S|$ and $S \subseteq \binom{T}{2}$ with $|T| = k$, this means that S contains all possible $\binom{k}{2}$ edges on the vertex set T . Therefore $G[T]$ is a clique on k vertices, and hence G contains a k -clique. This is sufficient to prove the claim. $\square$

Claim 4. *The construction above ensures that $k' = (k + 1) \binom{k}{2} \leq \frac{k^3}{2}$ and that H is $(k^2, 0)$ -nice.*

Proof. First point follows simply by definition of the reduction; indeed we have $k' = (k + 1) \binom{k}{2} \leq \frac{k^2 \cdot k}{2} = \frac{k^3}{2}$ for $k \geq 1$.

We now show that H is $(k^2, 0)$ -nice. By construction, every hyperedge of H is of the form

$$E_{uv} = Q_u \cup Q_v \cup R_e, \quad (34)$$

where $e = \{u, v\} \in E(G)$. Thus

$$|E_{uv}| = |Q_u| + |Q_v| + |R_e| = \binom{k}{2} + \binom{k}{2} + 1 = k(k - 1) + 1, \quad (35)$$

and hence

$$k(k-1) + 1 = k^2 - k + 1 \leq k^2 \tag{36}$$

for every $k \geq 1$. Therefore the rank of H (the maximum size of a hyperedge) is at most k^2 .

Consider the α -split of H with $\alpha := k^2$. Since no hyperedge has size greater than α , the upper part $H_{>\alpha}$ contains no hyperedges, and in particular every vertex has degree 0 in $H_{>\alpha}$. By the definition of (α, β) -niceness, this means that H is $(k^2, 0)$ -nice. $\square$

2 Additional Experiments

This section presents additional experimental results that complement the evaluation in Section 7. We extend the analysis along four main directions.

First, we provide further measurements of the build-up phase, including results for smaller hypergraphlet sizes ($k \in \{3, 4\}$), absolute running times, and a detailed breakdown of the build-up cost across datasets, which complement the speedup analysis reported in the main paper.

Second, we report a more detailed characterization of hypergraphlet frequency distributions, including the most frequent 4- and 5-hypergraphlets across all datasets.

Third, we include an extended scalability study that evaluates the parallel performance of HYPERMOTIVO across multiple datasets and values of k , covering a wider range of thread configurations and hypergraphlet sizes than those shown in the main paper.

Finally, we provide additional accuracy results, such as per-dataset error histograms and metrics for different sampling budgets and values of k , which refine and corroborate the empirical findings reported in Section 7.

Overall, these experiments offer a more complete picture of the performance, (parallel) scalability, and accuracy of hypergraphlet counting with HYPERMOTIVO.

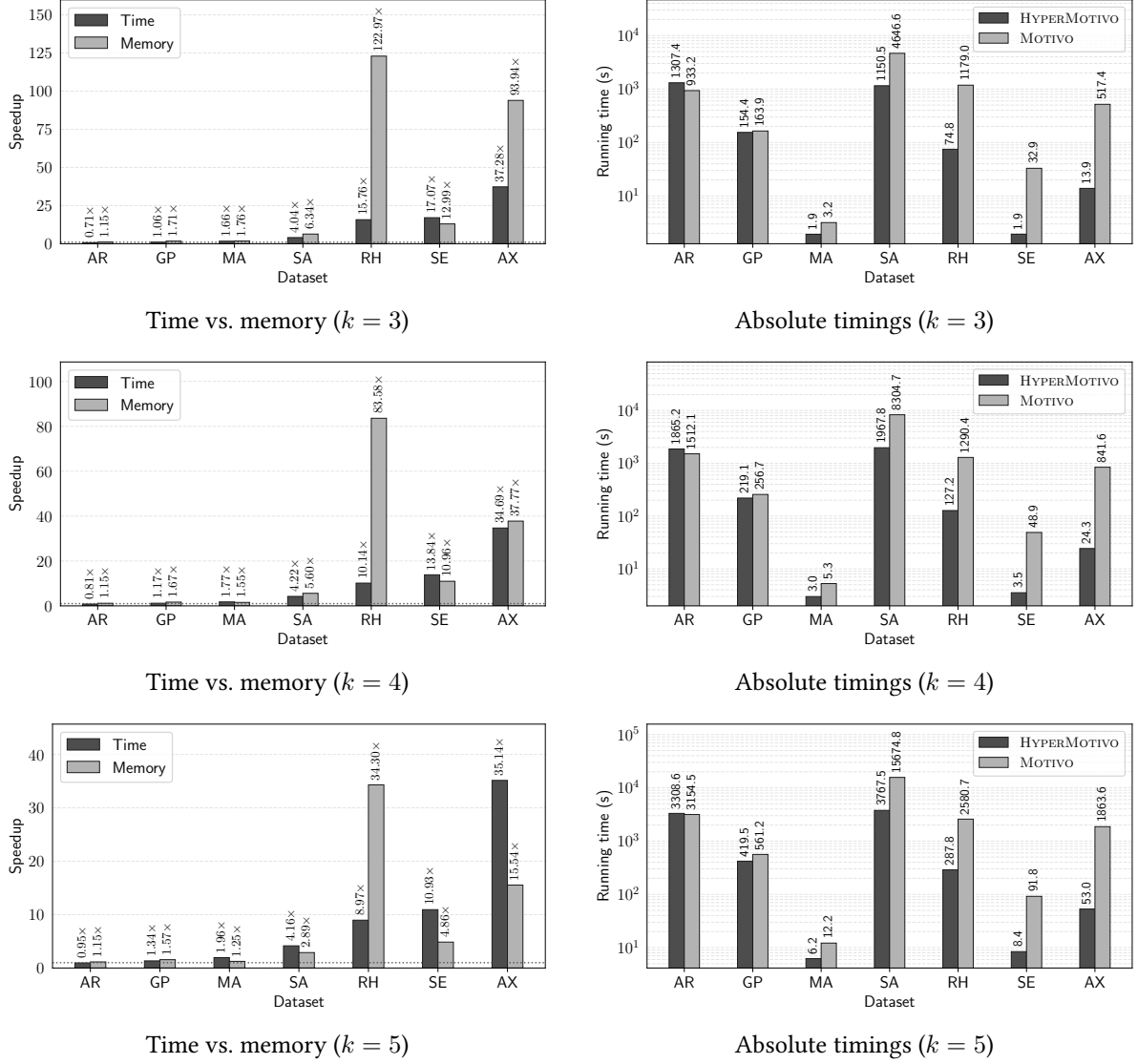


Figure 2: Memory-time tradeoff and absolute build-up timings for $k \in \{3, 4, 5\}$

2.1 Speedup and absolute build-up timings

Figure 2 shows, on the left, the speedup computed across all datasets. In addition to the case $k = 5$, which is already reported in the paper (see Section 7.2.2), we also include results for $k \in \{3, 4\}$. As observed in the paper, the speedup is consistent across different values of k . Higher values of k are computationally too demanding for the largest graphs. The right side of Figure 2 instead reports the absolute running times. Figure 3 reports absolute timing breakdown for each dataset and $k \in \{3, 4, 5\}$. For every dataset, the left group of bars reports the preprocessing and build-up times of MOTIVO, whereas the right group reports the preprocessing, build-up, and inclusion-exclusion times of HYPERMOTIVO. For MOTIVO, preprocessing includes projecting the input hypergraph H to its Gaifman graph $\text{Gaif}(H)$. For HYPERMOTIVO, preprocessing includes selecting α , splitting H into $H_{\leq \alpha}$ and $H_{> \alpha}$, and projecting $\text{Gaif}(H_{\leq \alpha})$.

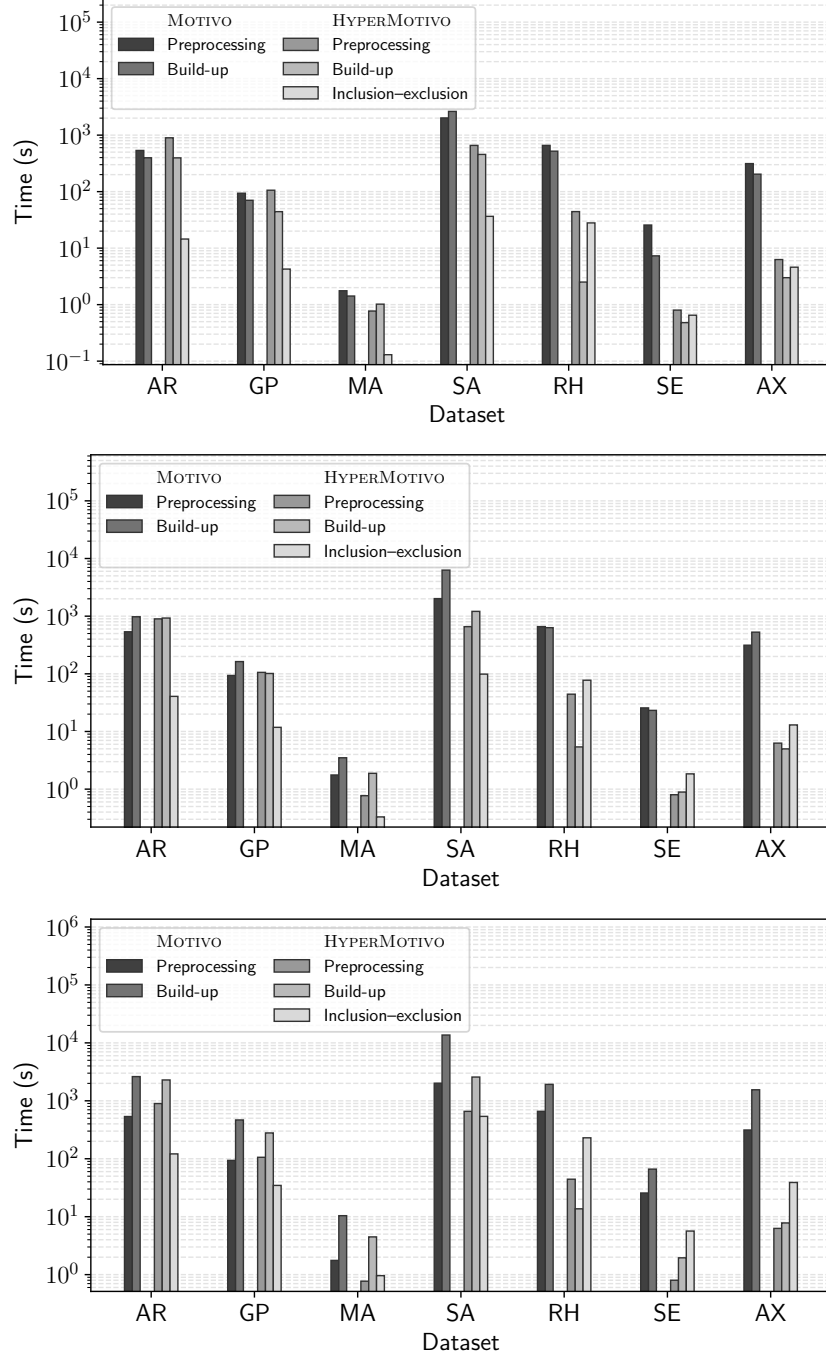


Figure 3: Timings breakdown for $k \in \{3, 4, 5\}$.

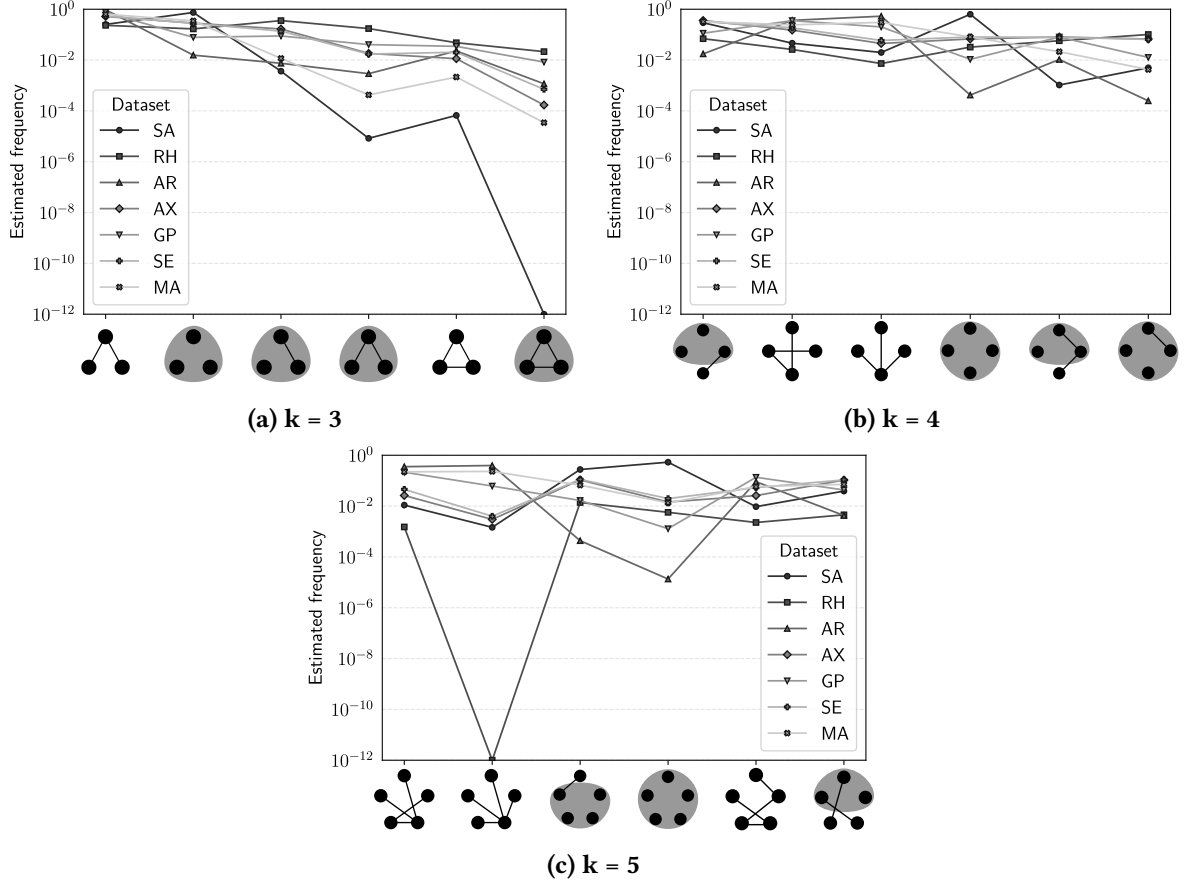


Figure 4: Frequency distributions of k -hypergraphlets for $k \in \{3, 4, 5\}$.

2.2 Hypergraphlet Frequencies

Figure 4 shows frequency distributions of 3, 4 and 5-hypergraphlets with highest aggregate frequency across all hypergraphs. Figure 5 reports the frequencies of the 20 most frequent k -hypergraphlets for $k \in \{4, 5\}$ across all datasets.

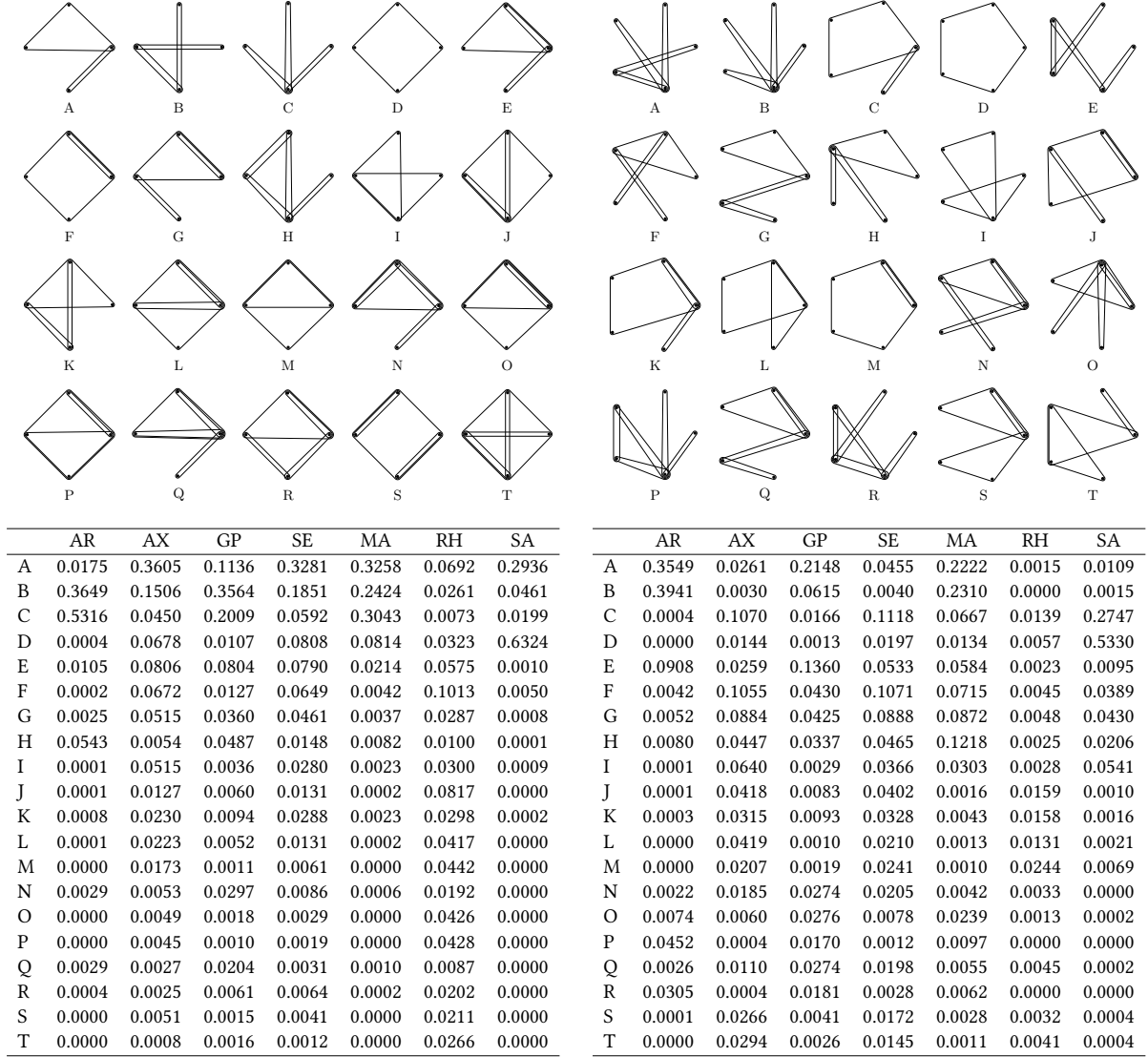


Figure 5: First 20 4- and 5-hypergraphlets with highest aggregate frequency across all datasets. Left: K_4 . Right: K_5 .

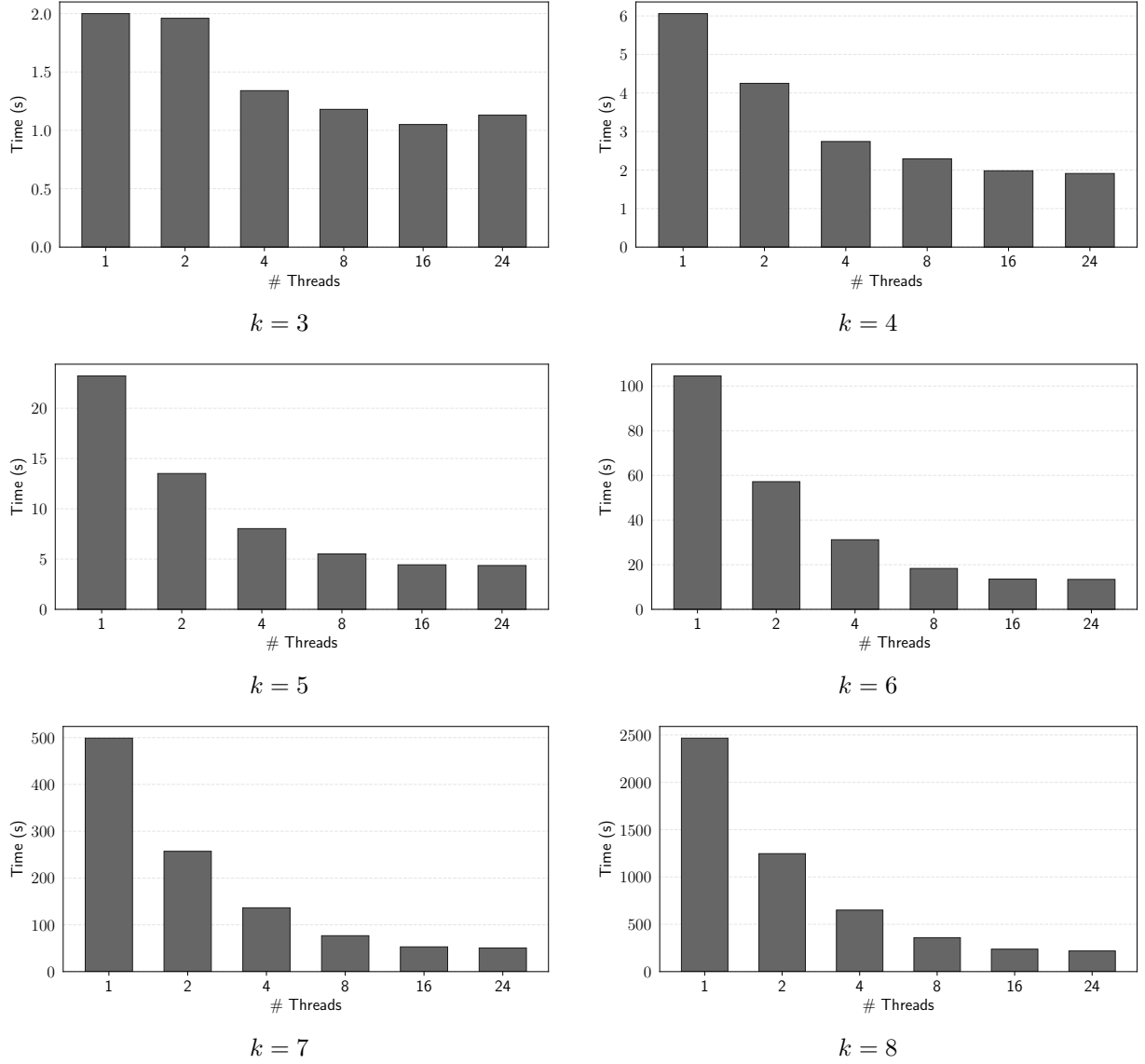
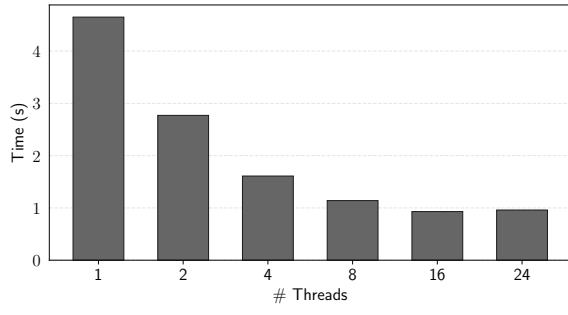


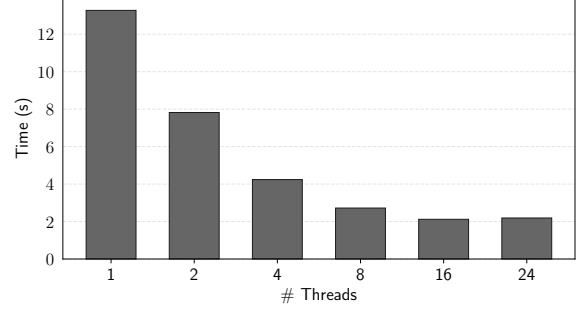
Figure 6: Running time of HYPERMOTIVO's build-up phase on the MA dataset for $k \in \{3, \dots, 8\}$.

2.3 Parallel scalability

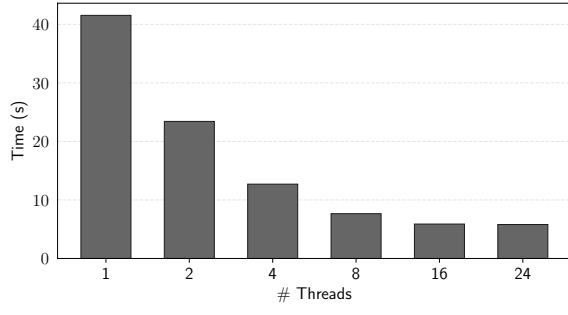
Here we present parallel scalability experiments for the datasets MA (Figure 6), SE (Figure 7), and AX (Figure 8), for $k \in \{3, \dots, 8\}$. We observe that scalability is consistent across all values of k and across all datasets. In some cases, when using 24 threads, a slight performance degradation is observed compared to 16 threads, likely due to communication overhead.



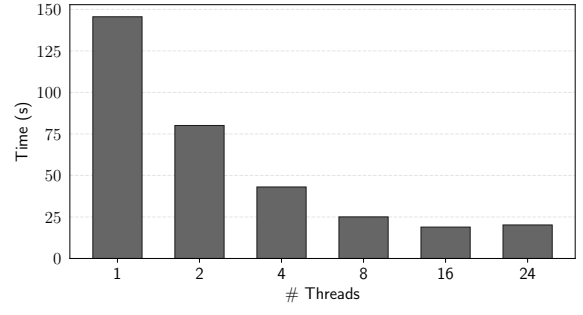
$k = 3$



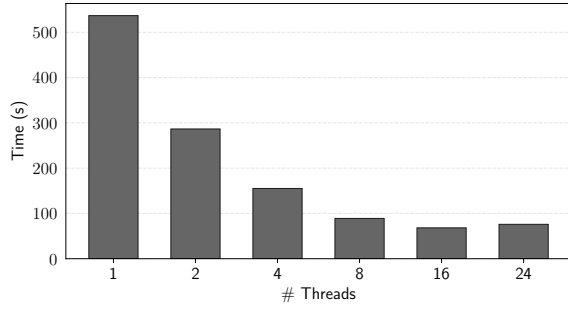
$k = 4$



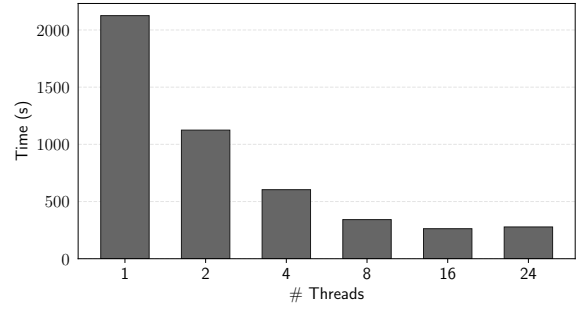
$k = 5$



$k = 6$

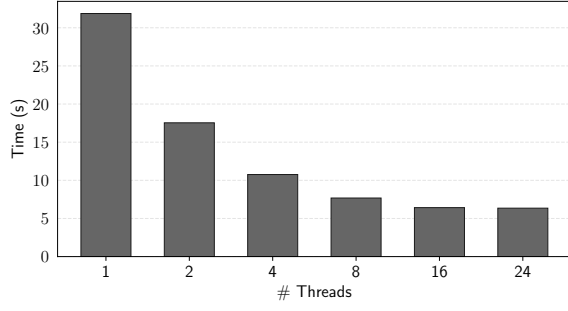


$k = 7$

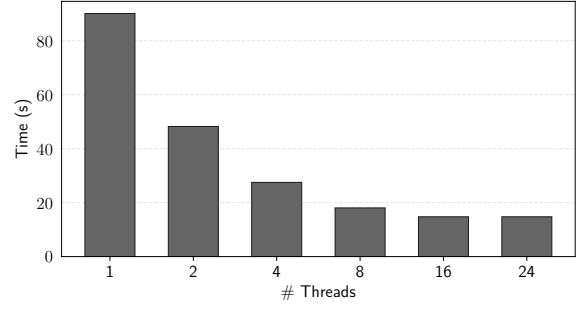


$k = 8$

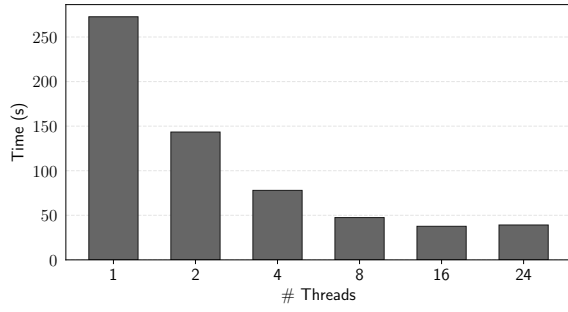
Figure 7: Running time of HYPERMOTIVO's build-up phase on the SE dataset for $k \in \{3, \dots, 8\}$.



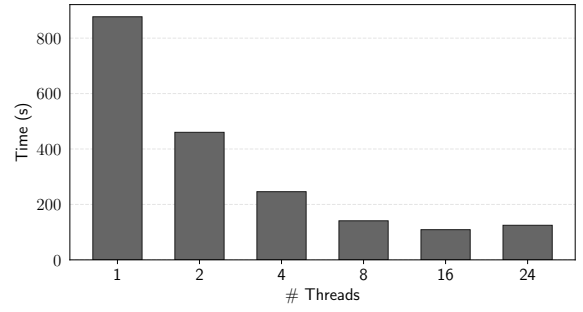
$k = 3$



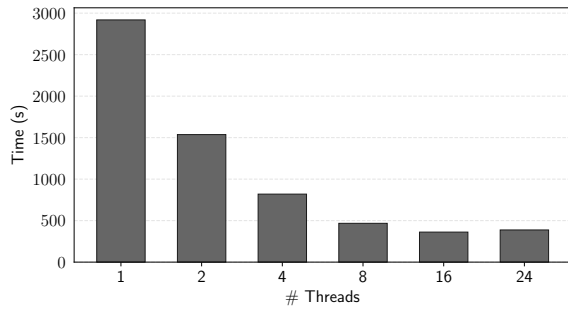
$k = 4$



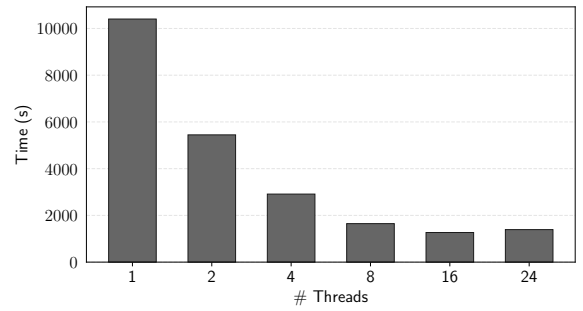
$k = 5$



$k = 6$



$k = 7$



$k = 8$

Figure 8: Running time of HYPERMOTIVO's build-up phase on the AX dataset for $k \in \{3, \dots, 8\}$.

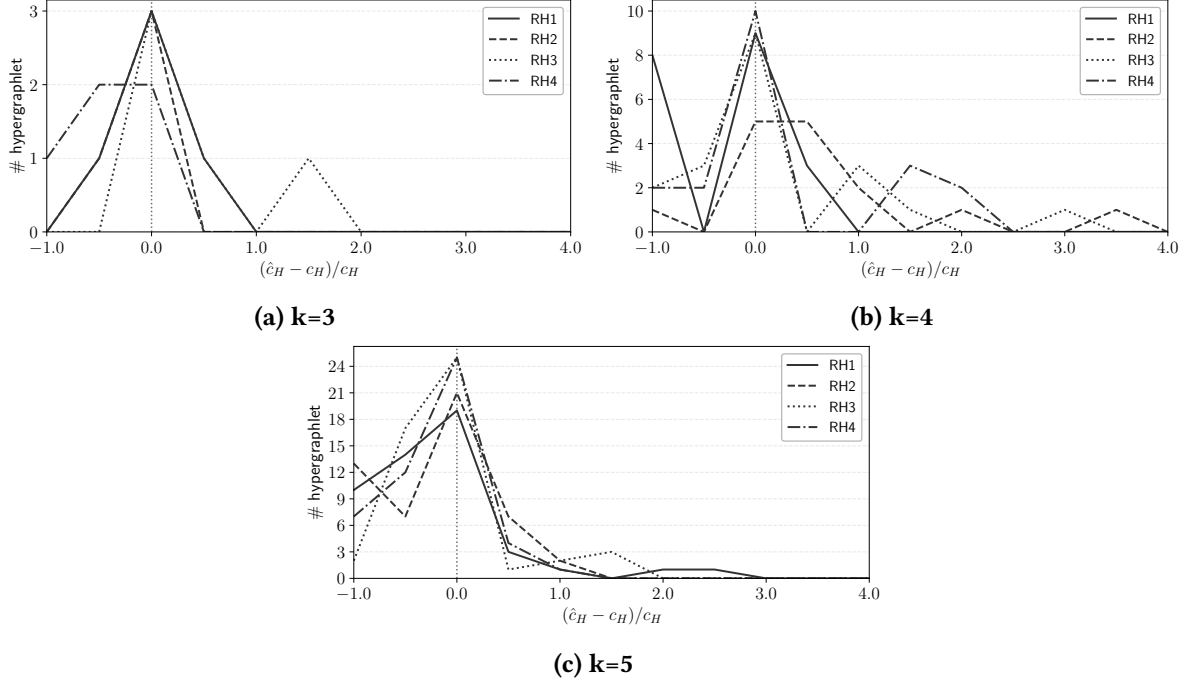


Figure 9: err_H distributions for $k \in \{3, 4, 5\}$ and 10^5 samples on four synthetic datasets.

2.4 Accuracy

We empirically assess the accuracy of HYPERMOTIVO on small synthetic hypergraphs, following the same protocol as in the main paper. For each hypergraphlet isomorphism type H , we report the relative count error $err_H = (\hat{c}_H - c_H)/c_H$, where c_H is the exact induced count of H in the input hypergraph and $\hat{c}_H$ is the estimate produced by HYPERMOTIVO using 10^5 samples. Figure 9 summarizes the resulting error distributions for $k \in \{3, 4, 5\}$ via histograms with bins of the form $[err - 0.25, err + 0.25)$.

For $k = 5$, the plot is restricted to the 50 most frequent hypergraphlet types by exact frequency, to avoid that extremely rare types, whose estimates are inherently high-variance under sampling, dominate the tails and obscure the typical behavior. For $k = 3$ and $k = 4$, instead, we include all observed types, at most 6 and 20, respectively. Despite the smaller number of types, their frequency distribution is still highly skewed: a few motifs account for the vast majority of occurrences, while the remaining ones are very rare. Consequently, even for $k = 3$, the overall error distribution is affected by these low-frequency types, which are more likely to be missed or to exhibit larger relative errors.